



A Research Agenda for the Living SysML V2 Blueprint: Toward Executable, Verifiable, and Navigable System Models

Ciprian Teodorov^{1,4(✉)}, Lucas Lima², Sidney C. Nogueira², Sylvain Guerin^{3,4},
and Loïc Lagadec^{1,4}

¹ ENSTA — Institut Polytechnique de Paris, Palaiseau, France
{ciprian.teodorov,loic.lagadec}@ensta.fr

² Universidade Federal Rural de Pernambuco, Pernambuco, Brazil
{lucas.albertins,sidney.nogueira}@ufrpe.br

³ IMT Atlantique, Nantes, France
sylvain.guerin@imt-atlantique.fr

⁴ Lab-STICC CNRS UMR 6285, Brest, France

<https://www.ensta.fr> , <https://www.ufrpe.br> , <https://www.imt-atlantique.fr>,
<https://labsticc.fr>

Abstract. Model-Based Systems Engineering (MBSE) has made substantial progress in managing system complexity, yet it still lacks principled support for understanding system behavior during early design. SysML v2 introduces a richer semantics for modeling behavior, but its utility remains limited by a lack of executable semantics, weak interoperability, and the absence of native support for formal verification and design-space exploration. We argue that a paradigm shift is needed, one in which behavioral models become first-class, executable artifacts, navigable within a dynamic design multiverse.

This position paper presents the vision for the Living SysML v2 Blueprint, a next-generation SysML v2 virtual machine that unifies dynamic execution, multiverse exploration, and native formal verification. Central to this vision is the Transparent Execution, Observation, and Control (TEOC) API, enabling accurate semantic inspection and tool interoperability without model transformation. By tolerating incomplete models and exposing structured execution traces, the LivingBlueprint VM empowers system engineers, verification experts, and business strategists to engage with design decisions iteratively, rigorously, and early.

This architecture redefines the semanticsverificationexecution triad at the heart of MBSE, offering new foundations for explainability, trust, and AI-augmented design. This paper outlines the theoretical pillars, practical challenges, and community-wide opportunities of this approach and invites collaborative efforts toward a new class of MBSE platforms centered around behavioral fidelity and multiverse awareness.

Keywords: SysML v2 · Execution Semantics · Formal Verification

1 Introduction

System Engineering (SE) has gone through a paradigm shift where traditional document-centric processes are being replaced by model-based initiatives. In this context, Model-based System Engineering (MBSE) has emerged as the *future of SE* given the supposed benefits it provides [27]. MBSE advocates that the “system model” is the unique source of truth, and it reflects the status and state of the system development [39]. It addresses design complexity by using abstract models for formal analysis, overcoming the limitations of document-based methods. It supports earlier Verification and Validation (V&V) of system design through the early and continuous availability of models [11].

The choice of the modeling language to support system engineering processes is crucial because it must sustain not only a common understanding of the system but also allow automated reasoning to optimize engineering tasks. The System Modeling Language (SysML v1) [51] stands out among these languages because it has achieved widespread adoption for large-scale systems in several domains, including automotive, avionics, and medicine [8]. A significant factor for its success was the good quality of tooling support from different vendors, which allowed, in addition to system design, the analysis, simulation, and code generation of those models. In fact, SysML is not only one but a collection of graphical languages that enable the design of different aspects of the system, including structures and behaviors. Despite its recognition by the system engineering community, it has several limitations that prevent it from increasing its reach. Poor semantic consistency, lack of interoperability mechanisms, and complex extension procedures are among its issues. To tackle these challenges, SysML v2 [41] has been proposed in two beta releases (2023 and 2024), promising increased precision, expressiveness, and interoperability, particularly through a new API and its textual language.

SysML v2 is still an evolving language. Its formal semantics is being constantly updated. Since its release, a pilot implementation of the language was made available in both Eclipse and Jupyter notebook environments with basic features [5]. To the best of our knowledge, tooling support for executing and analyzing the behavior of models is not provided by the pilot implementation or any other tool so far. However, we know that this was a relevant aspect that leveraged the adoption of its predecessor. Without an environment where models can be properly executed, simulated, debugged, and analyzed, the SysML v2 language is doomed to fail the community’s expectations. Hence, we plan to overcome these challenges by building the Living SysML v2 Blueprint to enact and verify SysML v2 models using state-of-the-art technologies and the work of experts on SysML semantics and tooling development. The Living Blueprint proposes a virtual machine (VM) that would encode the semantics for SysML v2 model elements, especially the ones related to behavior. Interactions with this VM will allow executing and navigating models using refined mechanisms, like multiverse exploration. Furthermore, we envision a verification module to enable the formal verification of properties directly against the model. Finally, we conceive that external communication with this environment can be achieved by a Transparent

Execution, Observation and Control (TEOC) API, facilitating interoperability with users and external tools. Therefore, this paper describes how we expect to concretize the Living SysML v2 Blueprint, describing its main features, principles, and challenges ahead. With this initiative, we envision that the future of MBSE must be execution- and verification-centric, with first-class support for multiverse exploration.

The remainder of this paper is structured as follows. Section 2 describes the current scenario for SysML v1 and v2, and their limitations. Section 3 details our proposal for the Living SysML v2 Blueprint, while Sect. 4 discusses its key principles. Section 5 presents the challenges we foresee in the path ahead, and Sect. 6 discusses the expected impact and opportunities. Finally, Sect. 7 shows research topics for potential collaborators, and Sect. 8 summarizes our vision and concludes.

2 Current State of the Art and Limitations

The adoption and evolution of Systems Modeling Language (SysML) have been shaped by a long-standing community effort to bridge high-level design with rigorous analysis. SysML v1 [51] achieved widespread use in domains like automotive and avionics, largely due to growing interest in MBSE and robust tooling support from various vendors. However, its success was hampered by fundamental limitations: poor semantic consistency, complex extension procedures, and a lack of standardized interoperability. This forced tool vendors to embed their own interpretations to provide analysis and simulation, effectively locking users into proprietary, semantically divergent, ecosystems.

The quest to overcome these limitations has a rich history. Early integrated environments, such as TOPCASED [19, 24], demonstrated the feasibility of federating UML/SysML editors with simulation [13] and formal analysis tools like Tina [6], OBP [16] or CADP [24]. However, this was achieved through intermediate languages and model transformations, which introduced semantic drift and significant maintenance overhead. In parallel, the rCOS Modeler [30] pioneered multi-view modeling with formal contracts and CSP-based consistency checks, advancing refinement reasoning but remaining software-centric and dependent on external provers like FDR.

This lineage of live, executable environments ranges from foundational systems like Smalltalk [25] to domain-specific instances like the *Play-In/Play-Out* approach for scenario specification [26]. Its subsequent evolution, with *Smart Play-Out*, further integrated formal verification and state-space exploration directly into the live environment. LivingBlueprint aims to bring the full power of this paradigm (characterized by transparency, introspection, and direct manipulation) to MBSE, scaling it to the heterogeneous context of SysML v2, making multiverse exploration and native verification first-class capabilities across all its behavioral formalisms.

This lineage reveals a persistent pattern: the reliance on model transformations as the primary means to enable execution and verification. While this

strategy made advanced analysis possible, it came at a cost. Each transformation step embeds implicit semantic assumptions that require fragile, labor-intensive, and often unverified equivalence proofs [1, 29, 50]. This complicates traceability, as counterexamples must be reverse-mapped across encodings, and often degrades property fidelity via semantic mismatches [1]. Furthermore, it creates a high barrier to adoption, as engineers must cope not only with complex domains and SysML itself but also with the intricacies of downstream formal methods tools.

SysML v2 [41] was proposed to overcome these challenges, moving away from a UML profile to a new foundation based on the Kernel Modeling Language (KerML) and its declarative semantics [2]. It introduces a textual notation for improved interoperability and a Systems Modeling API for tool integration. However, while its declarative semantics allows reasoning about model correctness, a standardized operational semantics, defining how the interpretations are executed is not yet available. Consequently, tooling support remains in its infancy. The current SysML v2 toolchain, including the Pilot Implementation¹ and initiatives like SysIDE [53], focuses primarily on model authoring, syntactic checks, and visualization. To the best of our knowledge, no environment supports the direct execution, simulation, or debugging of behavioral models. Recent research, such as the work by Molnár et al. [40], represents a crucial step toward the formal verification of SysML v2 models, but it still relies on a transformation-based pipeline, inheriting the associated challenges of semantic gaps and proof obligations.

This gap is also recognized industrially. While companies like Dassault Systèmes (with CATIA Magic) and initiatives like SysON² (Obeo, Eclipse Foundation) and EasyMOD project³ (IRT Saintexupery) are making strides in methodology, authoring, and usability, they primarily focus on the front-end of the modeling process. Consequently, they lack a native, verification-friendly execution core for SysML v2, ultimately falling back on the transformation-based paradigms that risks to perpetuate the very bottlenecks identified.

The lessons from this lineage are clear. Transformation-based approaches were a necessary stepping stone, but their inherent brittleness and complexity limit scalability and rigor. This has catalyzed *transformation-free verification*, a distinct research direction that directly reinforces our proposition [7, 29], where verification engines interact directly with model execution semantics rather than relying on intermediate translations. Recent advances in live modeling and integrated verification [18] exemplify this evolution, demonstrating the feasibility of unifying execution and model checking within a single semantics-aware runtime. Prior works on UML [7], AnimUML [28] and industrial BPMN tools [10] has consistently confirmed the value of avoiding translation chains and supporting the execution of partial models. Emerging tools like SysMD [14] further validate

¹ <https://github.com/Systems-Modeling/SysML-v2-Pilot-Implementation>.

² <https://mbse-syson.org/>.

³ <https://www.irt-saintexupery.com/project-easymod/>.

this shift by demonstrating the benefits of intuitive, iterative environments that lower the barrier to entry.

Therefore, a critical gap remains unaddressed in the current SysML v2 landscape, specifically: the absence of an environment where its behavioral models can be directly executed, navigated, and formally verified without intermediary transformations. The Living SysML v2 Blueprint vision is conceived to fill this gap. It integrates the lessons of the past by unifying execution, verification, and multiverse exploration within a single semantics-first virtual machine. Rather than maintaining separate proof obligations for every analysis pipeline, we envision a single verified execution substrate that centralizes correctness and enable early, live verification.

3 The Living SysML V2 Blueprint Vision

We envision a paradigm shift in model-based systems engineering: one where behavioral models are no longer passive artifacts, but active, executable structures that support rigorous analysis, early decision-making, and seamless collaboration. At the core of this vision is LivingBlueprint, a next-generation virtual machine for SysML v2 that unifies dynamic execution, multiverse exploration, and native formal verification, without requiring model transformations or syntactic compromises.

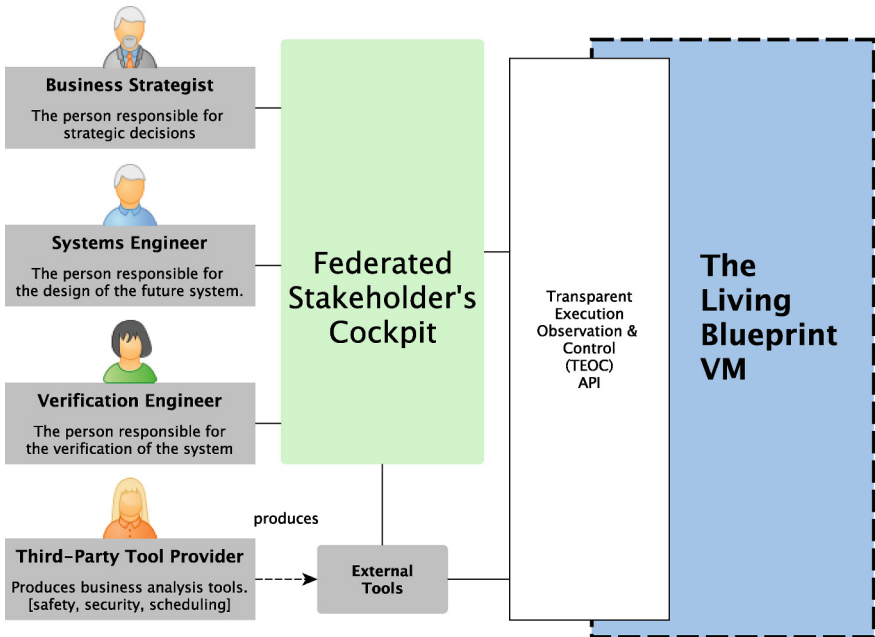


Fig. 1. High-Level Architecture of the Living SysML v2 Blueprint VM.

The high-level architecture of the Living SysML v2 Blueprint VM is shown in Fig. 1. The Living SysML v2 Blueprint VM is designed to serve the needs of four key stakeholder classes (left in Fig. 1), each interacting with the system at different levels of abstraction and with distinct objectives. Business strategists engage with the system at a higher level, using behavioral exploration tools to navigate, query, and analyze the space of possible executions, for scenario-based decision-making and “what-if” analyses that assess design trade-offs. System engineers require an intuitive execution framework that allows them to understand and refine behavioral models as they are being constructed, ensuring that early design decisions align with intended system behaviors. Verification engineers, on the other hand, demand rigorous formal validation capabilities to certify that behavioral specifications conform to logical correctness criteria, captured with temporal logic. Finally, third-party tool providers seek a robust and interoperable execution platform that allows them to integrate advanced analysis tools without relying on syntax-level model transformations or having to reimplement SysML v2’s dynamic semantics, thereby lowering the cost and complexity of external tool development.

The Living SysML v2 Blueprint Prototype is designed to offer a structured interaction model tailored to each stakeholder class, ensuring seamless integration with their workflows. System engineers, verification engineers, and business strategists interact with the VM through the Federated Stakeholder’s Cockpit (green box in the center of Fig. 1), a user interface built using model federation technology, leveraging the OpenFlexo [4]. This cockpit is designed to act as a bridge between their respective needs (i.e. design validation, behavioral exploration, and formal verification) and the TEOC API of the VM (white box in Fig. 1). By federating models and unifying access to different analytical perspectives, the cockpit would enable engineers to test, refine, and verify their models in a cohesive environment without requiring deep expertise in formal methods. Third-party tool providers primarily interact directly with the TEOC API, bypassing the cockpit to integrate external tools with minimal dependency. However, they can also leverage the federation-based user interface, ensuring a unified experience for integrating domain-specific extensions. This architecture ensures modularity, extensibility, and interoperability, allowing the LivingBlueprint VM to serve as a flexible and future-proof execution and verification platform for SysML v2.

The core of the Living SysML v2 Blueprint Prototype (blue box, right side in Fig. 1) would be built around 3 tightly integrated modules, each fulfilling a critical role in the execution, verification, and exploration of SysML v2 behavioral models. (1) At its foundation lies the Dynamic Semantics Module, which serves as the unique and authoritative source of interpretation for the SysML v2 behavioral subset. It ensures that all executions conform precisely to the intended semantics of SysML v2, avoiding ambiguities or deviations introduced by external transformations. This module provides the execution backbone for the VM, ensuring a faithful and consistent runtime environment. As SysML v2 behavior description is structured on actions and states, we will use our team’s expertise

on defining semantics for activities [38], state machines [20], and their integration [21] when specifying this module. (2) The Verification Module is intended to embed a customized version of the retargetable OBP2 verification engine [49], enabling formal property validation directly within the VM. By operating at the same semantic level as the execution engine, this module ensures high-fidelity verification results that align tightly with the model’s intended behavior. (3) Complementing these, the Multiverse Explorer Module is a high-performance behavior navigation engine built on the pioneering temporal multiverse debugging [42]. This module allows system engineers and business strategists to efficiently navigate, analyze, and query possible execution paths within the behavioral design space. By enabling rapid “what-if” scenario evaluations and behavioral insights, traditional software debugging becomes a structured exploration of specification’s behaviors. Together, these three modules form a cohesive, high-fidelity execution environment, making LivingBlueprint a groundbreaking platform for design-space exploration, verification, and behavioral understanding of SysML v2 models.

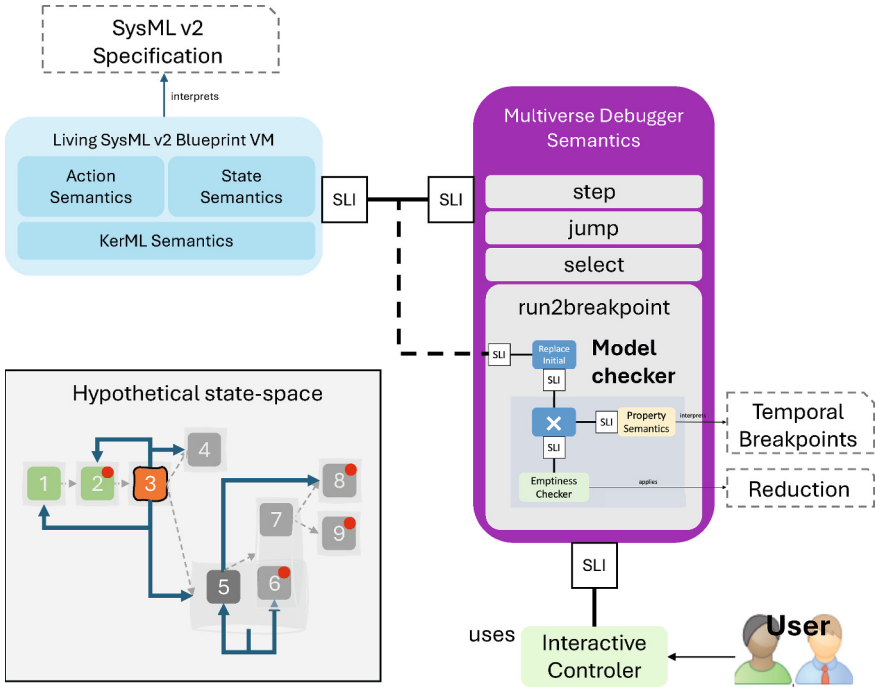


Fig. 2. Internal Architecture of the core of the Living SysML v2 Blueprint Virtual Machine (VM). The VM exposes a Semantic Language Interface through which the pioneering temporal multiverse debugger semantics [42] can be connected to efficiently navigate the possible execution paths. The Hypothetical state-space illustrates the navigation capabilities (blue arrows) enabled on a non-deterministic behavior.

The internal architecture of the core of the VM is shown in Fig. 2. The Dynamic Semantics Module composes the actions and state semantics based on the underlying KerML execution model. This module captures the SysML v2 dynamic semantics and exposes a Semantic Language Interface [42] (an essential part of the TEOC API) that offers a semantic bridge to build the analysis tools. The Verification Module and the Multiverse Explorer Module can both be realized by leveraging a state-of-the-art retargetable multiverse debugger [42], which offers a practical and scalable continuum between interactive debugging (interactive exploration of the behavioral state-space) and temporal logic model-checking. This Multiverse Debugger Semantics embeds the OBP2 verification engine [49] natively as well as a highly customizable model-reduction framework able to capture numerous under-approximation strategies. Consider, for instance, the Hypothetical state-space, illustrated in Fig. 2, and suppose that the current state reached during the navigation is state 3 (the orange-filled box). The debug actions exposed to the user are: (I) the possibility to interactively **step** through the state space. In this case, the user is presented with a non-deterministic choice between two targets (the state 4, and the cluster $\langle 5, 6 \rangle$), to which he can **step**. Say he steps to the cluster $\langle 5, 6 \rangle$, this cluster (instead of a single target state) represents the potential execution result of a non-deterministic action in the SysML v2 semantics⁴, which leads either to state 5 or to state 6; (II) the possibility to interactively **select** one of the states in a target cluster, state 5 or 6, when in the cluster $\langle 5, 6 \rangle$; (III) the possibility to **jump** back to any previously discovered state, either on the current path (the current universe) or another (an alternative universe that was previously unfolded). The availability of the branching history is one of the particularities of multiverse debugging (as opposed to omniscient debugging, which relies on a linear history structure). (IV) the possibility to **run to a breakpoint** (run2breakpoint in Fig. 2), which performs a model-checking reachability query (potentially reduced, using a user-specified **reduction**) driven by the breakpoint semantics (itself captured as an independent semantic component). The second important particularity of the multiverse debugger, used here, stems from the high expressivity of the breakpoints, which range from simple state-based invariants to rich temporal logic formulas. Through this setup, the typical temporal logic model-checking reduces to a run-to-breakpoint action executed in the initial states of the SysML v2 model with the added benefit of having counter-example traces encoded as sequences of native values in the underlying SysML v2 semantic domain (there is no need to transform the counter-examples back to SysML v2 values, as is the case in transformation-based model-checking). Furthermore, the Multiverse Debugger Semantics functions as a high-performance behavior navigation engine, allowing

⁴ One can argue that the dynamic semantics of SysML v2 should maybe be deterministic, which is definitely possible, however, SysML v2 is a specification language, and as such it should offer abstraction power, allowing non-deterministic execution provides that. Furthermore, the SysML v2 library-based approach to domain-specific language (DSL) embedding definitely opens the door to semantics with non-deterministic execution (consider, for instance, a DSL allowing non-deterministic assignment).

the interactive navigation of the state-space to further the understanding of the behaviors allowed by a SysML v2 specification. This architectural design underscores an innovative proposition of native integration of formal verification and design-space exploration directly within the VM, without introducing semantic brittleness and inconsistencies.

This section demonstrates how the Living SysML v2 BluePrint can potentially fill the gap by providing an environment that enables behavioral models to be directly executed, navigated, and formally verified without intermediary transformations. The SysML v2 virtual machine unifies dynamic execution, multiverse exploration, and native formal verification. The Dynamic Semantics Module would provide a well-defined interpretation of the SysML v2 behavioral subset, serving as a consistent execution environment for SysML v2 models. The Multiverse Explorer Module would offer an engine for exploring, analyzing, and querying possible execution paths within the behavioral design space. The Cockpit would serve as a bridge between behavioral exploration, formal verification, and the TEOC API of the virtual machine, while the TEOC API itself would enable precise semantic inspection and tool interoperability without requiring model transformation.

We strongly believe that building a SysML v2 virtual machine following these design ideas could be a cornerstone in system engineering. The approach we present here is more than a technical solution, it is a statement of intent: that models should live, evolve, and be trusted; that semantics should be explicit and navigable; and that system design should be deeply connected to system understanding. This vision redefines how we think about MBSE tools, not as containers of syntax, but as thinking environments, where design exploration, validation, and traceability emerge naturally from execution.

4 Key Principles

Here we lay down the principles that should guide the development of the Living SysML v2 Blueprint environment.

A Unified Execution and Verification Infrastructure. The central idea behind LivingBlueprint is to provide a semantics-first virtual machine capable of executing the behavioral subset of SysML v2 models directly and faithfully. Rather than treating execution and verification as post-processing steps, they are embedded within the runtime environment itself. This enables temporal logic properties to be checked in situ, supporting the analysis of non-deterministic behaviors, and facilitates early incremental feedback, even in the presence of incomplete or non-conformant models.

Multiverse Exploration as a Modeling Primitive. System engineers often face uncertainty not just in implementation, but in design intent, requirements evolution, and operational assumptions. LivingBlueprint proposes to introduce multiverse exploration as a first-class modeling capability, enabling engineers and

business strategists to explore possible execution paths, evaluate alternative scenarios, and understand design implications in a structured and interactive way. Inspired by advances in multiverse debugging and temporal logic model checking [42], this capability transforms debugging from a reactive task into a proactive design-space navigation tool.

The Transparent Execution, Observation, and Control API. A key principle is that LivingBlueprint would expose a Transparent Execution, Observation, and Control (TEOC) API, intended to allow external tools to interact with model execution at the semantic level. This API eliminates the need for brittle model transformations or proprietary hooks, offering a principled interface for integrating AI assistants, runtime monitoring tools, domain-specific verifiers, and federated model environments. It serves as a semantic bridge, not just between tools, but between communities: systems engineers, formal methods researchers, and tool developers.

Executable Semantics for a Continuum of Design. Unlike conventional MBSE platforms that require full specification before execution, LivingBlueprint supports partial and evolving models. This capability empowers engineers to simulate early behaviors, refine them over time, and use the same infrastructure from exploratory modeling to deployment validation. The platform thereby becomes a foundation for semantic-level digital twins, where high-level design models serve as seeds for runtime models linked through refinement mappings to physical implementations. This enables true model continuity, spanning design, analysis, deployment, and adaptation.

5 Scientific and Engineering Challenges

To advance the utility and rigor of SysML v2, several scientific challenges must be addressed. We classify these challenges in three categories: (1) semantic challenges, particularly the formalization and operationalization of the operational semantics; (2) verification challenges, particularly scalability, reuse-based abstract interpretation, heterogeneous refinement, and SysML-level proof engineering; and (3) interoperability.

5.1 Semantic-Level Challenges

High-Performance verification-Ready VM. Building a verification environment that provides results within an acceptable timeframe is crucial to facilitate adoption. This is not an easy task due to the complexity of current systems. The **key scientific challenge** lies in *establishing a formally grounded approach for high-performance operational semantics design*. A guiding promising direction is the Omnisemantics approach [12], which generalizes structural operational semantics to facilitate reasoning about non-determinism. One bridge between this theoretical semantics framework and practical virtual machine design is

established through the Semantic Language Interface (SLI) [50], which operationalizes Omnisemantics to integrate execution and formal behavioral analysis. Another venue for addressing this challenge is investing in meta-frameworks like K-framework [46], and Gemoc [55] to reduce their execution overhead and formal verification capabilities. To address the execution overhead, a SysML v2 virtual machine will need to integrate just-in-time code generation [17] and VM implementation techniques that allow for peak performance [34]. Today, it is not necessarily clear how these techniques compose with the needs of formal verification tools. Moreover, the SysML v2 behavioral analysis tools will have to provide the capabilities and performances offered by industry-ready [7, 10, 23], retargetable verification toolkits like LTSmin [29] or OBP [49]. By leveraging the $G\forall\text{min}\exists$ SLI [50], as presented in this position paper, we could, as a community, design a state-of-the-art VM for SysML v2 that combines meta-compilation techniques [34] with retargetable verification toolkits [49], all while adhering to a formally grounded approach for operational semantics design [12].

Why Not Transformations. In transformation-centric flows, each step embeds implicit semantic assumptions that must be proved semantics-preserving, proofs that are fragile under language/tool evolution and costly to maintain. They also complicate traceability (counterexamples require reverse mapping across encodings) and often degrade property fidelity via semantic mismatches [1]. By contrast, a single, verified execution substrate avoids duplicated proof obligations and keeps counterexamples and observables natively in the SysML v2 semantic domain.

As discussed in Sect. 2, our design proposes centralizing correctness into a single verified language VM. Any operational semantics introduced for SysML v2 must ultimately be shown correct with respect to the language’s formal semantics. However, our proposal aims to minimize the number of such correctness proofs that the ecosystem must maintain. In the current landscape, each model transformation or analysis tool implicitly embeds its own semantic assumptions, requiring separate validation. In contrast, by concentrating the executable meaning of SysML v2 into a single, verified virtual machine (VM), we shift from “many small, unverified transformations” to a single semantics-bearing execution substrate. Once this VM is proved sound and adequate with respect to the formal semantics, all analyses and integrations performed through it inherit that correctness by construction. Thus, the verification effort is centralized and reusable rather than duplicated across tools.

In practice, early versions of the VM may still rely on code generation or interfacing to external simulators for performance reasons. In such cases, the usual translation proof obligations remain; however, in some cases, the effort can be reduced through Pnueli’s translation validation technique [44]. Yet the long-term objective is to minimize transformation chains: whenever the VM can execute the model directly, it should, avoiding unnecessary re-encoding of the semantics. For non-software systems, where parts of the specification must still be deployed into the physical world, transformation will remain unavoidable, but it should be treated as a boundary operation, justified and verified only

where strictly necessary. Ultimately, proving the VM correct should be a one-time investment that can be maintained alongside its evolution, and potentially, in the future, the VM itself could be extracted from mechanized proofs, following approaches explored in verified compilers [36] and verified interpreters [9, 22].

Modular and Heterogeneous Semantic Composition. **Another key challenge** is *enabling modular and heterogeneous semantics design and composition at scale*. SysML v2, as its predecessor, is not one, but a set of several languages employed for different aspects of system engineering. When modeling behavior, it is possible to use both action-based and state-based behavior, and also combine them. Thus, building a semantics for SysML v2 that considers the composition of these two languages for describing behavior is paramount to give enough expressivity for system engineers. Furthermore, the library-based domain-specific language (DSL) embedding proposed in SysML v2 will give birth to a plethora of DSLs, which should compose with SysML v2 and benefit from the underlying SysML v2 execution infrastructure.

5.2 Formal Verification Challenges

Verification Scalability. Scalability is one of the factors that can impact performance, presenting both technical, engineering, and scientific challenges. The previously mentioned complexity of systems impacts the size of the state space to be traversed to check a property. Model checking and dynamic analysis methods often struggle with state-space explosion, making it infeasible to explore these complex systems. Therefore, researching the best methods and techniques to reduce scalability issues must be on the minds of our teams. We believe that not relying on transformations, like most formal methods-based strategies for MBSE verification, is an advantage of the environment presented in this paper. However, this approach should be coupled with model reduction strategies [43], which use under-approximation strategies to improve the scalability. The Living SysML v2 blueprint presented in the previous section integrates the support for a wide range of under-approximations: (1) generic reduction, such as bistrate hashing and hash compaction; (2) model-specific reduction, such as predicate abstraction; and (3) language-specific reductions that will be guided by the SysML v2 operational semantics. Nevertheless, *verification scalability* is **another key challenge** that should be addressed systemically in the context of SysML v2, and here we believe that abstract interpretation approaches hold the key.

Among the promising directions, we emphasize the foundational work on abstracting abstract machines [54], which demonstrates a systematic method for deriving static analyses from abstract machines through refactoring and store-allocated continuations. This approach shows how to transform machine semantics into an easily abstractable form to approximate machine behavior. This approach is particularly interesting in the context of SysML v2 due to the language's sheer size, which poses an enormous burden on the implementation of each abstract interpreter. By deriving abstract interpreters directly from the SysML v2 operational semantics, parts of the semantics can be reused to reduce

implementation effort. A derived approach dubbed abstracting definitional interpreters [15] highlights the reusability and extensibility of high-level abstract interpreters, particularly when structured with monad transformers, enabling the derivation of various program analyses by composing components. A key insight is the inheritance of properties like “pushdown control flow” from the metalanguage, simplifying complex analyses.

Recently, this approach was applied to WebAssembly to obtain modular definitional interpreters [9], which further demonstrates the ability to decompose the semantics of a complex language into language-independent components that can be instantiated later to obtain a highly configurable interpreter that efficiently offers whole program analysis support with minimal additional code. If we push these ideas further, in the context of SysML v2, we could get to the point where its operational semantics can be seen as a product line, with each variant being a VM geared to a particular usage context. Viewing the language semantics as a product line naturally brings the focus on establishing formal abstraction-refinement relations between the semantic variants.

Heterogeneous Refinement. Having an execution environment for analyzing abstract system specifications leads naturally to another **systemic challenge** *the use of heterogeneous refinement techniques to prove that a system correctly implements the specification*. In this case, the refinement is heterogeneous because the implementation execution infrastructure will not be the SysML v2 virtual machine itself, but an external execution environment that should be bound to the specification by refinement mappings prior to a joint analysis needed to prove the refinement. Working in this direction, implicitly, will offer support for using SysML v2 models as execution monitors and digital twins, even in cases where formal refinements are too difficult to prove or are not a goal of the system designer.

SysML v2 Proof Assistants. Sometimes, automated analysis techniques are not enough to cope with the verification, validation, and certification of complex systems. In these cases interactive proof assistants have been shown to help [35], by allowing the proof engineer to drive the proof efforts. The key challenge that we want to emphasize in this context is understanding the scope of the SysML v2 language and its limitations for proof engineering. In other words, is the SysML v2 library system powerful enough to facilitate the development of domain-specific proof assistants geared towards the system’s engineers’ needs? Another related challenge is understanding how the SysML v2 operational semantics, captured in a high-performance VM, can be exploited to serve as a bridge for external provers.

5.3 Interoperability

The last two paragraphs in the previous subsection, underlie another **key open challenge**: *how can the SysML v2 VM offer support for effective interoperability with the external world while preserving formal guarantees?* The research

efforts around reasoning about the foreign function interfaces of programming languages offer an interesting perspective on this question [52]. In software engineering the interoperability is typically handled through one of 3 approaches [33]: integration, where all components follow a standard structure - imposed by a standard, or an implementation language for instance; unification, where a common meta-level description structures the inter-operating components and their interactions; or federation, where the components are dynamically accommodated through a binding (adapter) model. Given the interoperability problems in systems engineering and the large variability of system analysis tools, we argue that a federation-based interoperability approach is necessary for SysML v2 to achieve a semantic-level interoperability that pushes past the limitations of syntactic-based interoperability (heavily relying on model transformation) and can offer strong correctness guarantees. While many model federation approaches have been proposed [3] (including Vitruvius [31], Epsilon [32], Reactive Links [45], comprehensive systems [47], and OpenFlexo [4]) none has yet considered in depth the rich relations that govern the interoperability of operational semantics (captured through a VM) and the numerous analysis tools in the system engineering toolkit.

We strongly believe that the unified execution and verification infrastructure proposed in this position paper can serve as a real-life experimental framework to help overcome some of these challenges and fuel both the scientific and engineering communities towards a new generation of system engineering tools.

6 Expected Impact and Opportunities

The LivingBlueprint vision holds the potential to reshape the foundations of Model-Based Systems Engineering by embedding execution, verification, and multiverse exploration at the heart of the modeling experience. Its anticipated impacts span scientific, industrial, and methodological domains.

6.1 Scientific Impact

LivingBlueprint redefines the semanticexecutionverification triad in MBSE by unifying them within a single executable environment. This enables rigorous reasoning on behavioral models without translation or semantic loss. By supporting the execution of incomplete or non-conformant models, it also opens new lines of inquiry into live modeling, behavioral analysis under uncertainty, and early design validation. Moreover, the integration of multiverse exploration introduces a structured way to reason about alternative system trajectories, pushing the boundary of what model execution can offer to engineering science.

An open, interoperable, high-performance virtual machine for SysML v2 presents a transformative opportunity for the scientific community. By enabling early execution, formal analysis, and multiverse exploration of behavioral models, LivingBlueprint provides a unique experimental platform where new ideas in semantics, verification, abstraction refinement, and dynamic system analysis can

be tested under realistic conditions. Unlike traditional academic prototypes, this infrastructure is designed for sustained interoperability and extensibility, allowing researchers to embed, evaluate, and scale their contributions within a meaningful systems engineering context. Beyond offering a rigorous foundation for theoretical exploration, LivingBlueprint serves as a direct conduit for scientific impact, enabling researchers to engage with pressing industrial challenges in safety, traceability, automation, and decision support. This dual role (as a research testbed and translational vehicle) positions LivingBlueprint to foster deeper collaborations between formal methods, software engineering, and MBSE communities, accelerating the evolution of system modeling from an abstract design discipline to a continuous, verifiable, and executable science.

6.2 Engineering and Industrial Opportunities

If realized, LivingBlueprint would introduce a transparent and verifiable execution backbone for SysML v2, potentially helping engineers catch specification errors early, explore alternatives systematically, and make decisions with semantic confidence. This improves the quality and agility of system design processes, with direct implications for high-stakes sectors such as aerospace, automotive, defense, and energy. The platform’s API-first architecture also lowers the barrier for third-party tool integration, encouraging a modular and interoperable ecosystem rather than monolithic toolchains.

Building on this transparent, semantics-first infrastructure, LivingBlueprint also opens the door to integrating foundation models [48], such as large language or multimodal models, as engineering assistants. Because all observable behaviors and properties are exposed through the TEOC API with explicit semantics, these models can safely operate on verified execution data to support engineers in natural-language querying, automated report generation, or design-space summarization. Rather than replacing formal reasoning, foundation models would amplify it: (1) translating complex traces into explanations, (2) proposing alternative configurations, and (3) assisting in requirements and specification refinement [37]. This combination of formal semantics and adaptive language models could significantly accelerate model comprehension and decision-making in industrial contexts.

6.3 Educational and Methodological Impact

By treating behavioral semantics as an active interface, LivingBlueprint helps demystify formal methods, lowering the learning curve for engineers and students alike. Its stakeholder-oriented cockpit and introspection tools support explainable modeling, making system behavior visible, navigable, and traceable across abstraction levels. This sets the stage for human-in-the-loop verification, collaborative design exploration, and AI-augmented modeling assistants grounded in observable model behavior.

6.4 Long-Term Opportunities

Looking ahead, LivingBlueprint lays the groundwork for a new class of MBSE platforms: *(a)* Semantics-aware development environments, where specifications are immediately executable and verifiable. *(b)* Standardized execution benchmarks, enabling meaningful comparison between SysML v2 tools and fostering open, community-driven validation processes. *(c)* Multiverse-native system design workflows, where exploration and verification are not add-ons, but first-class modeling activities. *(d)* Validation through refinement mappings, establishing traceable links between high-level behavioral models and their corresponding physical implementations, bridging the gap between specification and deployment. *(e)* Semantic-level Digital Twins and model continuity, where executable SysML v2 models become seeds for deploying digital twins. By maintaining refinement-based links across abstraction layers, LivingBlueprint enables continuity between design-time intent and runtime observability, opening new opportunities for system evolution, diagnosis, and predictive maintenance. *(f)* Explainable AI integrations, using structured behavior traces to train models for design synthesis, validation, or anomaly detection.

In essence, LivingBlueprint not only addresses immediate pain points in MBSE practice, it proposes a durable infrastructure for principled, participatory, and verifiable system design in an increasingly complex and interdependent world.

7 Call to Action

The LivingBlueprint vision is more than a proposal for a new tool; it is an invitation to reimagine the foundations of model-based systems engineering. As system complexity grows and trust in digital models becomes critical, we must equip engineers, researchers, and tool developers with infrastructures that are not only rigorous but also exploratory, explainable, and evolvable.

We call on the research community to recognize and contribute to LivingBlueprint as a semantic playground: a high-performance, interoperable execution platform where new ideas in modeling, verification, refinement, and dynamic analysis can be developed, tested, and validated on industrially relevant models. The open and modular architecture encourages contributions from formal methods, programming languages, AI, and MBSE researchers, turning the virtual machine into a shared foundation for cross-disciplinary innovation.

We also invite practitioners and system architects to explore a new kind of modeling workflow: execution-first, verification-aware, and decision-driven. By adopting semantic APIs and transparent behavioral introspection, LivingBlueprint lowers the barriers to early feedback, integration, and collaboration across stakeholder boundaries.

To sustain and expand this vision, we encourage the community to: *(1)* Contribute to the SysML v2 behavioral benchmark, enabling consistent evaluation of execution and verification platforms. *(2)* Experiment with semantic-level

model composition approaches, where the models of computation are interconnected natively instead of relying on long and brittle transformation chains. (3) Develop and share refinement-based validation strategies, bridging high-level design intent and deployed systems. (4) Co-design digital twin workflows grounded in semantic continuity, enabling verification-aware runtime observability. (5) Support the evolution of semantically grounded MBSE standards and tooling, advocating for model continuity, traceability, and trustworthy automation.

LivingBlueprint offers not only infrastructure, but direction: a path toward executable systems modeling grounded in science and aligned with engineering reality. We invite researchers, engineers, and toolmakers alike to join in building this path together.

The path toward explainable, evolvable, and trustworthy system models is open, but it must be built collaboratively. Let us not wait for tools to catch up with our ambitions. Let us define the future of MBSE by making its models live, introspective, and multiverse-aware.

8 Conclusion and Future Work

The increasing complexity and interdependence of modern systems demand a new generation of modeling infrastructures, ones that go beyond static diagrams and disconnected tools. LivingBlueprint answers this call by proposing a unified, executable, and verifiable environment for SysML v2 models. By embedding dynamic semantics, native formal verification, and multiverse exploration directly into the virtual machine, it repositions MBSE as a live, introspective, and explainable discipline.

We have argued that modeling must become a continuous act of observation and refinement, where system behavior is not inferred but experienced. The LivingBlueprint architecture, centered on the TEOC API, multiverse navigation, and partial model execution, supports this shift, enabling early feedback, semantic consistency, and stakeholder-inclusive workflows. It also provides the scientific community with a rare opportunity: a high-fidelity platform to explore new ideas in modeling, verification, refinement, and decision support, all grounded in a real engineering context.

LivingBlueprint is a call to reimagine how we think about models: not as passive documents, but as active agents in a larger epistemic and engineering process. By making models executable, traceable, and semantically transparent, we enable not only better systems but better system engineering.

Crucially, Sect. 3 provides an actionable stepping stone, the core internal architecture, that can be realized today, and which will de-risk the endeavor, deliver immediate value, and provide a solid foundation for the community to collaboratively tackle the challenges and incrementally realize the full vision.

The next step is community engagement. We invite researchers, practitioners, and toolmakers to join this effort, starting with this core VM, to help create a modeling ecosystem that is as dynamic, rigorous, and alive as the systems it seeks to build.

References

1. Ab. Rahim, L., Whittle, J.: A survey of approaches for verifying model transformations. *Softw. Syst. Model.* **14**(2), 1003–1028 (2013). <https://doi.org/10.1007/s10270-013-0358-0>
2. Almeida, J.a.P.A., Ferreira Pires, L., Guizzardi, G., Wagner, G.: An analysis of the semantic foundation of KerML and SysML v2. In: *Conceptual Modeling: 43rd International Conference, ER 2024, Pittsburgh, PA, USA, October 28–31, 2024, Proceedings*, pp. 133–151. Springer-Verlag, Berlin, Heidelberg (2024). https://doi.org/10.1007/978-3-031-75872-0_8
3. Amrani, M., et al.: A survey of federative approaches for model management in MBSE. In: *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, pp. 990–999 (2024)
4. Bach, J.C., Beugnard, A., Champeau, J., Dagnat, F., Guérin, S., Martínez, S.: 10 years of model federation with openflexo: challenges and lessons learned. In: *MODELS 2024: ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, pp. 12. ACM, Linz, Austria (Sep 2024). <https://doi.org/10.1145/3640310.3674084>, best Paper Award, Practice Track
5. Bajaj, M., Friedenthal, S., Seidewitz, E.: Systems modeling language (SysML v2) support for digital engineering. *Insight* **25**(1), 19–24 (2022)
6. Berthomieu, B., et al.: Fiacre: an Intermediate Language for Model Verification in the Topcased Environment. In: *4th European Congress ERTS Embedded Real Time Software (ERTS 2008)*, pp. 1–8. SEE : Société de l’électricité, de l’électronique et des technologies de l’information et de la communication and 3AF : Association Aéronautique et Astronautique de France, Toulouse, France (Jan 2008). <https://inria.hal.science/inria-00262442>
7. Besnard, V., Teodorov, C., Jouault, F., Brun, M., Dhaussy, P.: Unified verification and monitoring of executable UML specifications. *Softw. Syst. Model.* **20**(6), 1825–1855 (2021). <https://doi.org/10.1007/s10270-021-00923-9>
8. Bone, M.A., Cloutier, R.J.: The current state of model based systems engineering: results from the OMGTM SysML request for information 2009. In: *Proceedings of the 8th Conference on Systems Engineering Research* (2010)
9. Brandl, K., Erdweg, S., Keidel, S., Hansen, N.: Modular abstract definitional interpreters for webassembly. In: Ali, K., Salvaneschi, G. (eds.) *37th European Conference on Object-Oriented Programming (ECOOP 2023)*. Leibniz International Proceedings in Informatics (LIPIcs), vol. 263, pp. 5:1–5:28. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2023). <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2023.5>
10. Brumbulli, M., Gaudin, E., Teodorov, C.: Automatic verification of BPMN models. In: *10th European Congress on Embedded Real Time Software and Systems (ERTS 2020)*. Toulouse, France (Jan 2020). <https://hal.science/hal-02441878>
11. Cederbladh, J., Cicchetti, A., Suryadevara, J.: Early validation and verification of system behaviour in model-based systems engineering: a systematic literature review. *ACM Trans. Softw. Eng. Methodol.* (2023). <https://doi.org/10.1145/3631976>
12. Charguéraud, A., Chlipala, A., Erbsen, A., Gruetter, S.: Omnisemantics: Smooth handling of nondeterminism. *ACM Trans. Program. Lang. Syst.* **45**(1) (Mar 2023). <https://doi.org/10.1145/3579834>
13. Combemale, B., Crégut, X., Giacometti, J.P., Michel, P., Pantel, M.: Introducing simulation and model animation in the MDE Topcased Toolkit. In: *4th European*

- Congress EMBEDDED REAL TIME SOFTWARE (ERTS). <http://www.erts2008.org/> Toulouse, France (Jan 2008). <https://hal.science/hal-00371596>
14. Dalecke, S., Grimm, C.: Sysmd: an inclusive modelling tool. In: 2024 19th Annual System of Systems Engineering Conference (SoSE), pp. 178–183 (2024). <https://doi.org/10.1109/SOSE62659.2024.10620956>
 15. Darais, D., Labich, N., Nguyen, P.C., Van Horn, D.: Abstracting definitional interpreters (functional pearl). *Proc. ACM Program. Lang.* **1**(ICFP) (Aug 2017). <https://doi.org/10.1145/3110256>
 16. Dhaussy, P., Roger, J.C., Leroux, L., Boniol, F.: Context aware model exploration with obp tool to improve model-checking. In: ERTS 2012, p. xx. Toulouse, France (Feb 2012). <https://ensta.hal.science/hal-00676640>
 17. Ducasse, Q., Cotret, P., Lagadec, L.: Gigue: A JIT code binary generator for hardware testing. In: Proceedings of the 15th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages, pp. 73–82. VMIL 2023, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3623507.3623553>
 18. Exelmans, J., Teodorov, C., Vangheluwe, H.: Integrating model checking into a live modeling environment. In: Proceedings of the 18th ACM SIGPLAN International Conference on Software Language Engineering, pp. 128–133. SLE '25, Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3732771.3742718>
 19. Farail, P., et al.: The TOPCASED project: a Toolkit in Open source for Critical Aeronautic SysEms Design. In: ERTS 2006 proceedings. Toulouse, France (Jan 2006). <https://hal.science/hal-02270461>
 20. Ferreira, D., Lima, L.: A CSP semantics for UML state machines aiming at hidden formal methods verification. In: Formal Methods: Foundations and Applications: 27th Brazilian Symposium, SBMF 2024, Vitória, Brazil, December 4–6, 2024, Proceedings, pp. 49–67. Springer-Verlag, Berlin, Heidelberg (2024). https://doi.org/10.1007/978-3-031-78116-2_4
 21. Ferreira, D., Lima, L.: Verifying integrated designs of UML state machines and activities using CSP. In: C. Nogueira, S., Teodorov, C. (eds.) Formal Methods: Foundations and Applications, pp. 68–85. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-78116-2_5
 22. Franceschino, L., Pichardie, D., Talpin, J.P.: Verified functional programming of an abstract interpreter. In: Drăgoi, C., Mukherjee, S., Namjoshi, K. (eds.) Static Analysis, pp. 124–143. Springer International Publishing, Cham (2021)
 23. Gaudin, E., Brunel, E., Brumbulli, M.: Language agnostic model checking for SDL. In: 2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 231–240 (2023). <https://doi.org/10.1109/MODELS-C59198.2023.00052>
 24. Gauflillet, P., Farail, P.: TOPCASED results and benefits. In: ERTS 2012 Proceedings. Toulouse, France (Feb 2012). <https://hal.science/hal-02192082>
 25. Goldberg, A., Robson, D.: Smalltalk-80: The Language and its Implementation. Addison-Wesley Longman Publishing Co., Inc, USA (1983)
 26. Harel, D., Marelly, R.: Specifying and executing behavioral requirements: the play-in/play-out approach. *Softw. Syst. Model.* **2**(2), 82–107 (2003). <https://doi.org/10.1007/s10270-002-0015-5>
 27. Henderson, K., Salado, A.: Value and benefits of model-based systems engineering (MBSE): evidence from the literature. *Syst. Eng.* **24**(1), 51–66 (2021). <https://doi.org/10.1002/sys.21566>

28. Jouault, F., et al.: AnimUML: A practical tool for partial model animation and analysis. *Sci. Comput. Program.* **232**(C) (Jan 2024). <https://doi.org/10.1016/j.scico.2023.103050>
29. Kant, G., Laarman, A., Meijer, J., van de Pol, J., Blom, S., van Dijk, T.: LTSmin: high-performance language-independent model checking. In: Baier, C., Tinelli, C. (eds.) *TACAS 2015*. LNCS, vol. 9035, pp. 692–707. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46681-0_61
30. Ke, W., Li, X., Liu, Z., Stolz, V.: rCOS: a formal model-driven engineering method for component-based software. *Front. Comput. Sci China* **6**(1), 17–39 (2012)
31. Klare, H., Kramer, M.E., Langhammer, M., Werle, D., Burger, E., Reussner, R.: Enabling consistency in view-based system development – the vitruvius approach. *J. Syst. Softw.* **171**, 110815 (2021). <https://doi.org/10.1016/j.jss.2020.110815>
32. Kolovos, D., Paige, R., Polack, F.: Detecting and repairing inconsistencies across heterogeneous models. In: *International Conference on Software Testing, Verification, and Validation*, pp. 356–364 (2008). <https://doi.org/10.1109/ICST.2008.23>
33. Kosanke, K.: ISO standards for interoperability: a Comparison. In: Konstantas, D., Bourrières, J.P., Léonard, M., Boudjlida, N. (eds.) *Interoperability of Enterprise Software and Applications*, pp. 55–64. Springer, London, London (2006)
34. Larose, O., Kalebka, S., Burchell, H., Marr, S.: AST vs. Bytecode: interpreters in the age of meta-compilation. *Proc. ACM Program. Lang.* **7**(OOPSLA2) (Oct 2023). <https://doi.org/10.1145/3622808>
35. Lecomte, T., Deharbe, D., Prun, E., Mottin, E.: Applying a formal method in industry: a 25-year trajectory. In: Cavalheiro, S., Fiadeiro, J. (eds.) *Formal Methods: Foundations and Applications*, pp. 70–87. Springer International Publishing, Cham (2017)
36. Leroy, X.: Formal verification of a realistic compiler. *Commun. ACM* **52**(7), 107–115 (2009). <https://doi.org/10.1145/1538788.1538814>
37. Leung, M., Murphy, G.: On automated assistants for software development: the role of LLMs. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1737–1741 (2023). <https://doi.org/10.1109/ASE56229.2023.00035>
38. Lima, L., Tavares, A., Nogueira, S.C.: A framework for verifying deadlock and nondeterminism in UML activity diagrams based on CSP. *Sci. Comput. Program.* **197**, 102497 (2020). <https://doi.org/10.1016/j.scico.2020.102497>
39. Madni, A.M., Sievers, M.: Model-based systems engineering: Motivation, current status, and needed advances. In: Madni, A.M., Boehm, B., Ghanem, R.G., Erwin, D., Wheaton, M.J. (eds.) *Disciplinary Convergence in Systems Engineering Research*, pp. 311–325. Springer International Publishing, Cham (2018)
40. Molnár, et al.: Towards the Formal Verification of SysML v2 Models. In: *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, pp. 1086–1095. *MODELS Companion '24*, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3652620.3687820>
41. OMG: OMG Systems Modeling Language (SysML v2) - Part 1: Language Specification. <https://www.omg.org/spec/SysML/2.0/Beta2/Language/PDF> (2024)
42. Pasquier, M., Teodorov, C., Jouault, F., Brun, M., Le Roux, L., Lagadec, L.: Temporal breakpoints for multiverse debugging. In: *Proceedings of the 16th ACM SIGPLAN International Conference on Software Language Engineering*. pp. 125–137. *SLE 2023*, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3623476.3623526>

43. Pasquier, M., Teodorov, C., Jouault, F., Brun, M., Roux, L.L., Lagadec, L.: Practical multiverse debugging through user-defined reductions: application to UML models. In: Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems, pp. 87–97. MODELS '22, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3550355.3552447>
44. Pnneli, A., Shtrichman, O., Siegel, M.: Translation validation for synchronous languages. In: Larsen, K.G., Skyum, S., Winskel, G. (eds.) ICALP 1998. LNCS, vol. 1443, pp. 235–246. Springer, Heidelberg (1998). <https://doi.org/10.1007/BFb0055057>
45. Rațiu, C.C., Assunção, W.K., Herac, E., Haas, R., Lauwerys, C., Egyed, A.: Using reactive links to propagate changes across engineering models. *Softw. Syst. Model.* 1–27 (2024)
46. Rosu, G., Serbanuta, T.F.: An overview of the K semantic framework. *J. Logic Algebraic Programm.* **79**(6), 397–434 (2010). <https://doi.org/10.1016/j.jlap.2010.03.012>, membrane computing and programming
47. Stünkel, P., König, H., Lamo, Y., Rutle, A.: Comprehensive systems: a formal foundation for multi-model consistency management. *Formal Aspects Comput.* **33**(6), 1067–1114 (2021)
48. Sun, J., et al.: A survey of reasoning with foundation models: concepts, methodologies, and outlook. *ACM Comput. Surv.* **57**(11) (Jun 2025). <https://doi.org/10.1145/3729218>
49. Team, O.R.: OBP2 high-performance, retargetable temporal logic verification engine. <http://www.obpcdl.org/> (17 march 2025)
50. Teodorov, C.: Gʻvminᄁ: Exploring the Boundary Between Executable Specification Languages and Behavior Analysis Tools. Habilitation à diriger des recherches, Université de Bretagne Occidentale (UBO), Brest (Apr 2023). <https://hal.science/tel-04066483>
51. The Object Management Group: OMG Systems Modeling Language. Standard 1.7 beta, Object Management Group, Milford, MA, USA (2022). <https://www.omg.org/spec/SysML/1.7/Beta1>
52. Turcotte, A., Arteca, E., Richards, G.: Reasoning About Foreign Function Interfaces Without Modelling the Foreign Language. In: Donaldson, A.F. (ed.) 33rd European Conference on Object-Oriented Programming (ECOOP 2019). Leibniz International Proceedings in Informatics (LIPIcs), vol. 134, pp. 16:1–16:32. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2019). <https://doi.org/10.4230/LIPIcs.ECOOP.2019.16>
53. Vaicenavicius, J., Wiklund, T., Kavolis, D., Draukšas, S., Kalkauskas, A., Vaicevičius, R.: SysIDE: SysML v2 textual editing and analysis system: overview and applications. *CEAS Space J.* (02 2025). <https://doi.org/10.1007/s12567-025-00595-x>
54. Van Horn, D., Might, M.: Abstracting abstract machines. *SIGPLAN Not.* **45**(9), 51–62 (2010). <https://doi.org/10.1145/1932681.1863553>
55. Zschaler, S., Bousse, E., Deantoni, J., Combemale, B.: A generic framework for representing and analyzing model concurrency. *Softw. Syst. Model.* **22**(4), 1319–1340 (2023). <https://doi.org/10.1007/s10270-022-01073-2>